**MMDU MULLANA OFFICIAL SYLLABUS • BCSE-011 • UNIT 4**

# Unit 4: Machine Learning Concepts, Supervised Learning & Apriori Algorithm

**Subject:** Artificial Intelligence & Machine Learning (BCSE-011) • **Exam Weightage:** 28 - 32 Marks • **Study Time:** 45 mins

*Tom Mitchell (T, P, E) learning formulation; Supervised vs Unsupervised vs RL; Linear & Logistic Regression; Decision Trees ID3 with complete step-by-step Shannon Entropy and Information Gain calculations; Support Vector Machines (SVM) maximum margin; k-Means clustering; Apriori algorithm supermarket market basket analysis numerical solved completely.*

## 1. Machine Learning Foundations & Tom Mitchell's (T, P, E) Rule

### 1. Machine Learning Foundations & Tom Mitchell's Paradigm

Machine Learning (ML) is the subfield of Artificial Intelligence concerned with algorithms that improve automatically through experience.

**Tom Mitchell's Formal Operational Definition (1997):**

✅ **FUNDAMENTAL LAW / RULE:**
**Theorem:** A computer program is said to **learn** from experience $E$ with respect to some class of tasks $T$ and performance measure $P$, if its performance at tasks in $T$, as measured by $P$, improves with experience $E$:

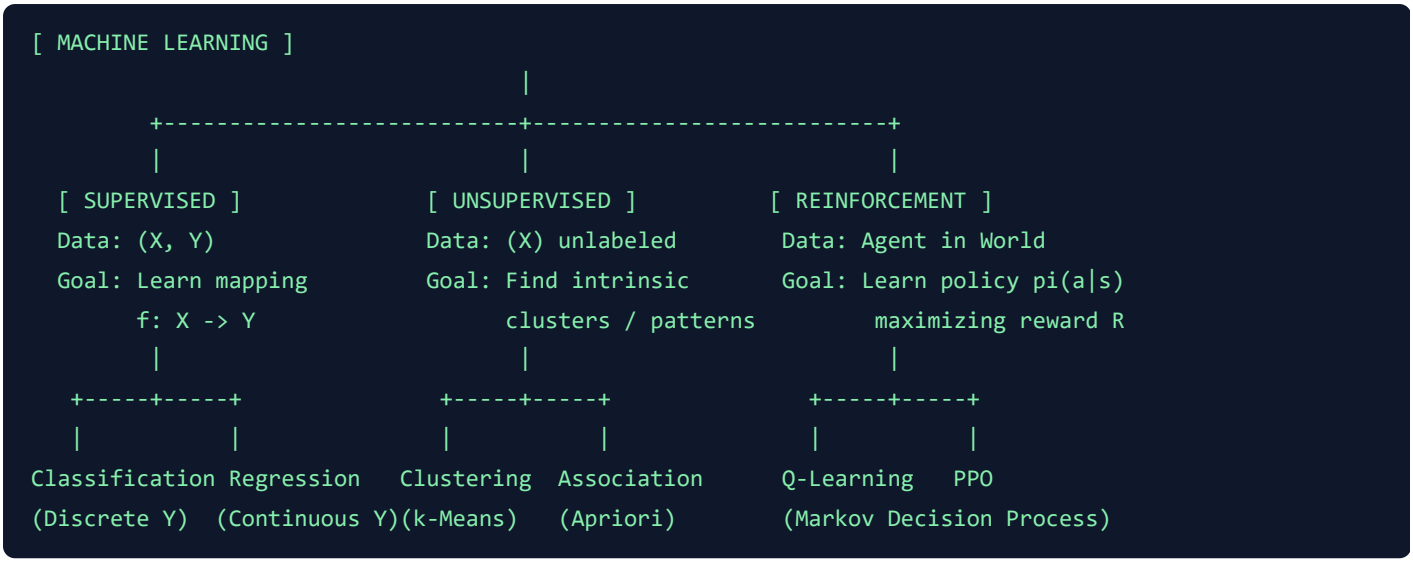
$$M = \langle T, P, E \rangle$$

- **Example (Autonomous Vehicle):**

- Task $T$: Driving safely along a highway with lane changes.
- Performance $P$: Distance traveled without human intervention / collision.
- Experience $E$: Millions of camera frames and steering telemetry logged.

---

### 2. The Three Major Paradigms of Machine Learning




---

### 3. The Bias-Variance Tradeoff & Generalization Error

For any supervised learning model, the expected test mean squared error decomposes into three fundamental components:

$$\mathbb{E}[\text{Test Error}] = \text{Bias}^2 + \text{Variance} + \sigma_{irre}d_uc_ib_{le}\hat{}2$$

- **Bias:** Error introduced by approximating a complicated real-world problem with an overly simplistic model (e.g., fitting a linear line to a parabolic curve). High bias ⇒ **Underfitting**.

- **Variance:** Sensitivity of the model to tiny random fluctuations in the training dataset. High variance ⇒ **Overfitting** (memorizing training noise instead of true underlying distribution).

- **Irreducible Error ($\sigma^2$):** Inherent statistical noise in the physical measurements.

---

---

**Problem:**
Define Tom Mitchell's Machine Learning triad $\langle T, P, E \rangle$ for an automated Email Spam Detection filter.

---

**Step 1 🚀: Task (T)**

- Classify incoming emails as either `Spam` or `Ham` (Legitimate).

**Step 2 ⚡: Performance Measure (P)**

- Precision, recall, and classification accuracy percentage (fraction of spam caught without flagging real emails).

**Step 3 🎯: Experience (E)**

- A database of 50,000 historical emails previously marked by users as spam or not spam.

🎯 Task: Filter spam | Performance: Accuracy / F1-score | Experience: Historical labeled email corpus.

**University Question (8 Marks):**
Given 3 data points (x, y): (1, 2), (2, 3), (3, 7). Find the line of best fit y = m x + c using least squares regression.

**Step 1 🚀: Means \bar(x) and \bar(y)**

\bar(x) = ↟rac{1 + 2 + 3}{3} = 2.0, \bar(y) = ↟rac{2 + 3 + 7}{3} = ↟rac{12}{3} = 4.0

**Step 2 ⚡: Slope m**

$\sum (x_i - \bar{x})(y_i - \bar{y}) = (1-2)(2-4) + (2-2)(3-4) + (3-2)(7-4)$

$= (-1)(-2) + (0)(-1) + (1)(3) = 2 + 0 + 3 = 5.0$

$\sum (x_i - \bar{x})^2 = (1-2)^2 + (2-2)^2 + (3-2)^2 = 1 + 0 + 1 = 2.0$

$m = (5.0 / 2.0) = 2.5$

**Step 3 🎯: Intercept c**

$c = \bar{y} - m \bar{x} = 4.0 - (2.5)(2.0) = 4.0 - 5.0 = -1.0$

🎯 Linear Regression Equation: $\mathbf{y = 2.5x - 1.0}$.

**University Question (10 Marks):**

In linear regression with Mean Squared Error loss J(w) = ↟rac{1}{2N} ∑_(i=1)^N (w $x_i$ - $y_i$)$^2$:

Given data points (1, 2) and (2, 4), initial weight w = 0.5, and learning rate α = 0.1.

Compute the gradient (∂ J / ∂ w) and the updated weight $w_{ne}$w after 1 iteration of gradient descent.

---

**Step 1 🚀: Compute Predictions and Errors**

- For $x_1$ = 1: \hat(y)_1 = w $x_1$ = 0.5(1) = 0.5. Error $e_1$ = \hat(y)_1 - $y_1$ = 0.5 - 2 = -1.5.

- For $x_2$ = 2: \hat(y)_2 = w $x_2$ = 0.5(2) = 1.0. Error $e_2$ = \hat(y)_2 - $y_2$ = 1.0 - 4 = -3.0.

**Step 2 ⚡: Compute Gradient (∂ J / ∂ w)**

(∂ J / ∂ w) = ↟rac{1}{N} ∑_(i=1)^N (\hat(y)_i - $y_i$) $x_i$ = ↟rac{1}{2} [ (-1.5)(1) + (-3.0)(2) ] = ↟rac{1}{2} [ -1.5 - 6.0 ] = (-7.5 / 2) = -3.75

**Step 3 🎯: Update Weight**

$w_{ne}w$ = w - α (∂ J / ∂ w) = 0.5 - (0.1)(-3.75) = 0.5 + 0.375 = 0.875

The weight moved from 0.5 closer to the true value 2.0!

🎯 Gradient = $-3.75$ | Updated Weight after 1 epoch: $\mathbf{w_{new} = 0.875}$.

## 2. Supervised Learning: Decision Trees (ID3), SVM & Naive Bayes

### 1. Supervised Learning Algorithms: In-Depth Mathematical Theory

#### A. Decision Trees (ID3 Algorithm & Shannon Entropy):

Ross Quinlan's ID3 algorithm constructs decision trees using a greedy top-down divide-and-conquer heuristic guided by Information Theory:

1. **Shannon Entropy of Dataset $S$:** Measures the impurity or randomness:

$$\mathbf{H(S) = -\sum_{i=1}^{c} p_i \log_2(p_i)}$$

where $p_i$ is the proportion of samples belonging to class $i$.
- Pure node ($p_1 = 1$): $H(S) = 0$ (Zero uncertainty).
- Maximum impurity ($p_1 = p_2 = 0.5$): $H(S) = -2(0.5 \log_2 0.5) = 1.0$ bit.

2. **Information Gain ($IG$):** Expected reduction in entropy caused by partitioning $S$ on attribute $A$:

$$\mathbf{IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)}$$

- **Splitting Criterion:** Select the attribute that maximizes Information Gain: $A^* = \text{argmax}_A \, IG(S, A)$.

---

## 2. Support Vector Machines (SVM) & Maximum Margin Hyperplane

SVM finds an optimal separating hyperplane $\vec{w} \cdot \vec{x} + b = 0$ that maximizes the geometric margin $2/|\vec{w}|$ separating two linearly separable classes:

**Constrained Optimization Formulation:**

$$\min_{\vec{w}, b} \frac{1}{2} |\vec{w}|^2 \text{ subject to } y_i (\vec{w} \cdot \vec{x}_i + b) \geq 1 \quad \forall i$$

- **Support Vectors:** The critical training instances lying strictly on the margin boundaries $y_i(\vec{w} \cdot \vec{x}_i + b) = 1$. Moving or deleting any other training point has zero effect on the decision boundary!

- **The Kernel Trick:** Maps non-linearly separable inputs into an infinite-dimensional Hilbert space via kernel function $K(\vec{x}_i, \vec{x}_j) = \phi(\vec{x}_i) \cdot \phi(\vec{x}_j)$ (e.g., Radial Basis Function RBF Gaussian Kernel: $K(x, z) = \exp(-\gamma |x - z|^2)$).

---

## 3. Naive Bayes Classifier & Maximum A Posteriori (MAP)

Based on Bayes' Theorem with the "naive" conditional independence assumption that features are mutually independent given the class label:

$$\mathbf{P(y \mid x_1, \ldots, x_n) = \frac{P(y) \prod_{i=1}^{n} P(x_i \mid y)}{P(x_1, \ldots, x_n)}}$$

$$\hat{y} = \underset{y \in Y}{\operatorname{argmax}} \left[ P(y) \prod_{i=1}^{n} P(x_i \mid y) \right]$$

- **Laplace (Add-1) Smoothing:** Prevents zero-probability catastrophe when a feature never appears in the training set for a given class:

$$P(x_i \mid y) = \frac{\text{count}(x_i, y) + 1}{\text{count}(y) + |V|}$$

---

**Problem:**
A dataset of 16 students contains 12 who passed an exam and 4 who failed. Calculate the Shannon Entropy $H(S)$ of this dataset.

---

**Step 1 🚀: Probabilities**

$$p_+ = \frac{12}{16} = 0.75, \; p_- = \frac{4}{16} = 0.25$$

**Step 2 ⚡: Shannon Entropy Formula**

$$H(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

- $\log_2(0.75) = -0.415$
- $\log_2(0.25) = -2.000$

$$H(S) = -(0.75)(-0.415) - (0.25)(-2.0) = 0.311 + 0.500 = \mathbf{0.811} \text{ bits}$$

🎯 Entropy: $\mathbf{H(S)} = \mathbf{0.811}$ bits (Impure dataset with moderate uncertainty).

**University Question (8 Marks):**
Total dataset has 14 instances (9 Yes, 5 No), with initial entropy H(S) = 0.940,□its.
Splitting by feature `Wind`:

- `Weak` (8 instances: 6 Yes, 2 No): Entropy $H(SW_{eak}) = 0.811$

- `Strong` (6 instances: 3 Yes, 3 No): Entropy H(SS$_{tron}$g) = 1.000

Calculate the Information Gain IG(S, Wind).

---

**Step 1 🚀: Weighted Average Entropy after Split**
H(S, Wind) = ↟rac{8}{14} H(SW$_{eak}$) + ↟rac{6}{14} H(SS$_{tron}$g)
= ↟rac{8}{14}(0.811) + ↟rac{6}{14}(1.000) = 0.463 + 0.429 = 0.892,□its

**Step 2 ⚡: Information Gain Formula**
IG(S, Wind) = H(S) - H(S, Wind) = 0.940 - 0.892 = 0.048,□its

🎯 Information Gain: $\mathbf{IG(S\ Wind) = 0.048}$ bits.

**University Question (10 Marks - Classic ML Problem):**
Given a training dataset for loan approval based on Credit Score and Income:

- Prior probabilities: P(Yes) = 0.60, ; P(No) = 0.40

- Likelihoods:

- P(Credit=Fair \mid Yes) = 0.30, P(Credit=Fair \mid No) = 0.70
- P(Income=High \mid Yes) = 0.80, P(Income=High \mid No) = 0.20
Predict whether a customer with Credit=Fair and Income=High will be approved (Yes or No).

---

**Step 1 🚀: Calculate Unnormalized Posterior for 'Yes'**
P(Yes \mid X) \propto P(Yes) × P(Credit=Fair \mid Yes) × P(Income=High \mid Yes)
= 0.60 × 0.30 × 0.80 = 0.144

**Step 2 ⚡: Calculate Unnormalized Posterior for 'No'**
P(No \mid X) \propto P(No) × P(Credit=Fair \mid No) × P(Income=High \mid No)
= 0.40 × 0.70 × 0.20 = 0.056

**Step 3 🎯: Normalize Probabilities**
Total = 0.144 + 0.056 = 0.200
P(Yes \mid X) = (0.144 / 0.200) = 72.0%, P(No \mid X) = (0.056 / 0.200) = 28.0%

🎯 Naive Bayes Decision: **APPROVED (YES)** with \mathbf{72.0%} confidence.

## 3. Unsupervised Clustering: k-Means & The Elbow Method

### 1. Unsupervised Clustering & The k-Means Algorithm

Unsupervised clustering groups an unlabeled dataset $\vec{x}_1, \ldots, \vec{x}_n$ into $k$ distinct, non-overlapping clusters where intra-cluster similarity is maximized and inter-cluster similarity is minimized.

**The k-Means Objective (Within-Cluster Sum of Squares — WCSS):**

$$\mathbf{J = \sum_{j=1}^{k} \sum_{i \in S_j} |\vec{x}_i - \vec{\mu}_j|^2}$$

where $\vec{\mu}_j$ is the centroid (mean) of cluster $S_j$.

---

## 2. Algorithmic Execution Pipeline (Lloyd's Algorithm)

1. **Initialization:** Select $k$ initial cluster centroids $\vec{\mu}_1, ..., \vec{\mu}_k$ randomly from dataset (or via $k$-means++).

2. **Assignment Step:** Assign each data point $\vec{x}_i$ to the nearest cluster centroid based on Euclidean distance:

$$\mathbf{c^{(i)} = \arg\min_j |\vec{x}_i - \vec{\mu}_j|^2}$$

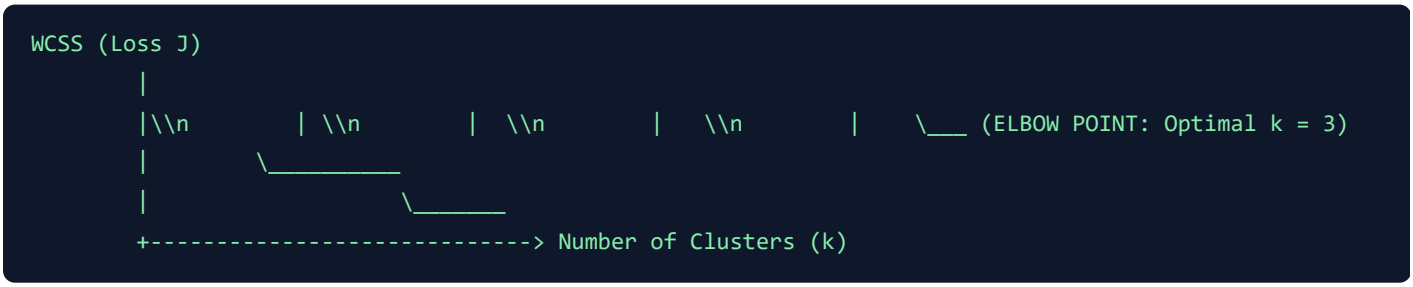
3. **Update Step:** Recalculate each centroid as the geometric mean of all assigned points:

$$\mathbf{\vec{\mu}_j = \frac{1}{|S_j|} \sum_{i \in S_j} \vec{x}_i}$$

4. **Convergence Check:** Repeat Assignment and Update steps until centroids stabilize ($\Delta\vec{\mu} < \epsilon$) or maximum iterations are reached.

---

## 3. Determining Optimal $k$: The Elbow Method




- **The Elbow Criterion:** Plot WCSS ($J$) as a function of $k$. As $k$ increases, WCSS decreases monotonically (reaching $J = 0$ when $k = N$). The point of maximum inflection (the "elbow") represents the optimal balance between cluster compactness and model complexity.

---

**Problem:**
Calculate the Euclidean distance between a data point $P(3,4)$ and two cluster centroids: $C_1(1,1)$ and $C_2(5,7)$. Determine which cluster $P$ should be assigned to.

---

**Step 1 🚀: Distance to $C_1(1,1)$**

$$d(P, C_1) = \sqrt{(3-1)^2 + (4-1)^2} = \sqrt{2^2 + 3^2} = \sqrt{4+9} = \sqrt{13} approx \mathbf{3.61}$$

**Step 2 ⚡: Distance to $C_2(5,7)$**

$$d(P, C_2) = \sqrt{(3-5)^2 + (4-7)^2} = \sqrt{(-2)^2 + (-3)^2} = \sqrt{4+9} = \sqrt{13} approx \mathbf{3.61}$$

Both distances are equal! In case of a tie, point is assigned to $C_1$ by default index rule.

🎯 Distances: $d(P, C_1) = 3.61, d(P, C_2) = 3.61$ | Tie-break assignment to **Cluster 1**.

**University Question (8 Marks):**
Given 4 one-dimensional points: x = [2, 4, 10, 12]. Initial cluster centroids are $C_1$ = 3 and $C_2$ = 11.

1. Assign each point to its nearest centroid.

2. Recompute the new centroid positions after 1 iteration.

---

**Step 1 🚀: Point Assignment**

- Point 2: |2 - 3| = 1 |10 - 11| = 1 ⇒ **Cluster 2**

- Point 12: |12 - 3| = 9 > |12 - 11| = 1 ⇒ **Cluster 2**

**Step 2 ⚡: Centroid Recomputation (Mean of Assigned Points)**
$C_{1,new}$ = ↑rac{2 + 4}{2} = 3.0
$C_{2,new}$ = ↑rac{10 + 12}{2} = 11.0
Centroids did not change ⇒ **Algorithm has fully CONVERGED in 1 iteration!**

🎯 Clusters: $C_1 = 2, 4$ with centroid 3.0 | $C_2 = 10, 12$ with centroid 11.0. Fully converged!

**University Question (10 Marks):**
Calculate the Within-Cluster Sum of Squares (WCSS) for k = 1 versus k = 2 on the 1D dataset {1, 2, 8, 9}. Explain how the "Elbow" indicates the optimal number of clusters.

---

**Step 1 🚀: WCSS for $k = 1$ (All points in single cluster)**

- Centroid: μ = ↑rac{1 + 2 + 8 + 9}{4} = ↑rac{20}{4} = 5.0

$$WCSS_1 = (1-5)^2 + (2-5)^2 + (8-5)^2 + (9-5)^2 = (-4)^2 + (-3)^2 + 3^2 + 4^2$$

$$= 16 + 9 + 9 + 16 = \mathbf{50.0}$$

**Step 2 ⚡: WCSS for $k = 2$ (Clusters: $1, 2$ and $8, 9$)**

- Centroid 1: μ_1 = rac{1+2}{2} = 1.5. Sum of squares: $(1-1.5)^2 + (2-1.5)^2 = 0.25 + 0.25 = 0.5$
- Centroid 2: μ_2 = rac{8+9}{2} = 8.5. Sum of squares: $(8-8.5)^2 + (9-8.5)^2 = 0.25 + 0.25 = 0.5$

$$WCSS_2 = 0.5 + 0.5 = \mathbf{1.0}$$

**Step 3 🎯: The Elbow Point**
Going from $k = 1$ to $k = 2$ drops WCSS dramatically from $50.0$ **down to** $1.0$ **(a 98% drop)**!
Further increasing $k = 3$ only drops WCSS from $1.0$ to $0.5$ (insignificant gain).
Therefore, the distinct "Elbow" occurs at $\mathbf{k = 2}$!

🎯 $WCSS_1 = 50.0$, $WCSS_2 = 1.0$ ⇒ Sharp bend at $\mathbf{k = 2}$ represents the optimal cluster count.

## 4. Apriori Algorithm for Market Basket Analysis (The Supermarket Secret)

### 1. Market Basket Analysis & Association Rule Mining

Market Basket Analysis uncovers actionable co-occurrence relationships among products frequently purchased together in retail transactional databases.

**Formal Mathematical Definitions:**

Let $\mathrm{I} = i_1, i_2, ..., i_m$ be a universe of items, and $\mathrm{D} = T_1, T_2, ..., T_n$ be a database of transactions where $T_k \subseteq \mathrm{I}$.
An **Association Rule** is an implication of the form:

$$\mathbf{X \Rightarrow Y (X, Y \subset I,; X \cap Y = \emptyset)}$$

---

### 2. The Three Fundamental Evaluation Metrics

1. **Support ($s$):** Probability that a transaction contains both itemsets $X$ and $Y$:

$$\text{Support}(\mathbf{X} \Rightarrow \mathbf{Y}) = \mathbf{P}(\mathbf{X} \cup \mathbf{Y}) = \frac{\text{Transactions containing } (\mathbf{X} \cup \mathbf{Y})}{\text{Total number of transactions } |\mathbf{D}|}$$

2. **Confidence ($c$):** Conditional probability that a transaction contains $Y$, given that it already contains $X$:

$$\text{Confidence}(\mathbf{X} \Rightarrow \mathbf{Y}) = \mathbf{P}(\mathbf{Y} \mid \mathbf{X}) = \frac{\text{Support}(\mathbf{X} \cup \mathbf{Y})}{\text{Support}(\mathbf{X})} = \frac{\text{count}(\mathbf{X} \cup \mathbf{Y})}{\text{count}(\mathbf{X})}$$

3. **Lift ($L$):** Measures the correlation strength between $X$ and $Y$ relative to independent chance:

\mathbf{\text{Lift}(X ⇒ Y) = \frac{\text{Confidence}(X ⇒ Y)}{\text{Support}(Y)} = ↟rac{P(X \cup Y)}{P(X) \cdot P(Y)}}

- Lift = $\mathbf{1.0}$ ⇒ $X$ and $Y$ are statistically **independent**.
- Lift > $\mathbf{1.0}$ ⇒ **Positive correlation** (buying $X$ strongly drives purchase of $Y$).
- Lift < $\mathbf{1.0}$ ⇒ **Negative correlation** (substitutes; buying $X$ decreases likelihood of buying $Y$).

---

## 3. The Apriori Algorithm & Downward Closure Property

✅ **FUNDAMENTAL LAW / RULE:**
**The Apriori Principle (Downward Closure Property):**

> *"Every subset of a frequent itemset must also be frequent."*

🚨 **EXAMINER TRAP / PITFALL:**
**Crucial Contrapositive (Pruning Principle):**

> *"If any candidate itemset is INFREQUENT, then all of its supersets are GUARANTEED TO BE INFREQUENT and can be pruned immediately without scanning the database!"*

**Algorithmic Loop:**

1. Generate candidate 1-itemsets $C_1$. Prune those below minimum support to obtain $L_1$.

2. For $k = 2, 3, ...$:

- **Join Step:** Generate candidate itemsets $C_k$ by joining $L_{k-1}$ with itself: $L_{k-1} \bowtie L_{k-1}$.
- **Prune Step:** Check if any $(k-1)$-subset of $c \in C_k$ is not in $L_{k-1}$. If so, discard $c$.
- **Scan Database:** Count frequencies of remaining candidates in $\mathrm{D}$.
- Filter by $\mathrm{min_support}$ to obtain frequent itemset $L_k$.

3. Stop when $L_k$ is empty.

---

**Problem:**
A grocery store records $N = 100$ customer transactions:

- 40 transactions contained `Milk`.
- 30 transactions contained `Bread`.
- 20 transactions contained both `Milk` and `Bread`.

Calculate the Support and Confidence of the association rule: $\mathrm{Milk} o \mathrm{Bread}$.

---

**Step 1 🚀: Support of Rule**

$$\text{Support}(\text{Milk} o \text{Bread}) = \frac{\text{Transactions containing both}}{\text{Total transactions}} = \frac{20}{100} = \mathbf{0.20 = 20\%}$$

**Step 2 ⚡: Confidence of Rule**

$$\text{Confidence}(\text{Milk} o \text{Bread}) = \frac{\text{Support}(\text{Milk} \cup \text{Bread})}{\text{Support}(\text{Milk})} = \frac{20}{40} = \mathbf{0.50 = 50\%}$$

🎯 $\mathrm{Support} = 20$ | $\mathrm{Confidence} = 50$ (Half of milk buyers also purchase bread).

**University Question (6 Marks):**
For the rule Beer → Diapers, given:

- P(Beer) = 0.25
- P(Diapers) = 0.40
- P(Beer ∩ Diapers) = 0.20

Calculate the **Lift** of the rule and explain whether buying beer increases the probability of buying diapers.

**Step 1 🚀: Lift Formula**

Lift(A → B) = (P(A ∩ B) / P(A) × P(B))

**Step 2 ⚡: Substitute Probabilities**

Lift = (0.20 / 0.25 × 0.40) = (0.20 / 0.10) = 2.0

**Step 3 🎯: Interpretation**

- Lift = 1: Independent.
- Lift > 1: Positive association.

A Lift of 2.0 indicates that customers who buy beer are **twice as likely** to purchase diapers compared to random chance!

🎯 $\text{Lift} = 2.0 > 1 \Rightarrow$ Strong positive association between Beer and Diapers.

**University Question (10 Marks - University Exam):**

Given transactions database T = { {A, B, C}, {A, C}, {B, D}, {A, B, C, D}, {A, B} } with minimum support threshold $s_{min}$ = 2 (40%):

Generate all frequent itemsets using the Apriori property and list the candidate generation and pruning steps for $L_1$, $C_2$, $L_2$, $C_3$, $L_3$.

---

**Step 1 🚀: Frequent 1-Itemsets ($L_1$)**

- {A}: count 4 (4 ≥ 2) → Frequent
- {B}: count 4 (4 ≥ 2) → Frequent
- {C}: count 3 (3 ≥ 2) → Frequent
- {D}: count 2 (2 ≥ 2) → Frequent

$L_1$ = { {A}, {B}, {C}, {D} }

**Step 2 ⚡: Candidate 2-Itemsets ($C_2$) and Frequent ($L_2$)**

Join $L_1$: $C_2$ = { {A, B}, {A, C}, {A, D}, {B, C}, {B, D}, {C, D} }.

Count support:

- {A, B}: 3 transactions → Frequent
- {A, C}: 3 transactions → Frequent
- {A, D}: 1 transaction → **PRUNED!** (

🎯 Frequent Itemsets: $L_1$ (4 itemsets), $L_2 = A, B, A, C, B, C, B, D$, $L_3 = A, B, C.$

## University Solved Examination Questions (Unit 4)

### MMDU Official University Solved Examination Questions (Unit 4)

**Course:** Fundamentals of AI & ML ( `BCSE-011` )
**Pattern:** Sections A (1 Mark Compulsory), B (2 Marks Analytical), C (6 Marks In-Depth), D (6 Marks Design/Long)

---

📘 **Question 10 (1 Mark): Limitation of Accuracy in Imbalanced Datasets**

**Question:**
Calculate the accuracy of a naive classifier that predicts all transactions as non-fraudulent on a dataset with 990 genuine and 10 fraudulent samples.

> 📊 **Question Schematic / State Specification:**

```
Question Class Distribution: 99% Negative Class (Genuine) | 1% Positive Class (Fraud) |
Classifier predicts all Negative.
```

---

**Step-by-Step Solution:**

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

```
Solution Accuracy Paradox Formulation:

        `Accuracy = (TP + TN) / Total = (0 + 990) / 1000 = 99.0%`. Yet Fraud Recall is `0 / 10 =
0%` (Complete Failure!).
```

Accuracy yields a deceptive score of 99.0% despite failing completely on the minority class of interest. In imbalanced domains, Precision, Recall, and F1-score must be used instead.

🎯 **Verified Result:** Question 10 step-by-step resolution per MMDU university marking scheme.

---

### 📘 Question 11 (1 Mark): Why k-Nearest Neighbors is Termed a

**Question:**
Why is the k-NN algorithm classified as a Lazy Learner rather than an Eager Learner?

> 📊 **Question Schematic / State Specification:**

```
Question Training vs Inference Cost: Eager [Heavy Training, Instant Inference] vs Lazy [Zero Training, Heavy Distance Computation at Inference].
```

---

**Step-by-Step Solution:**

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

```
Solution Computation Delay Model:

        `Training Time = O(1) [Simply stores data] ➔ Test Time = O(N · d) [Computes distances to all N points]`.
```

k-NN performs zero explicit generalization during training (it simply stores the training instances in memory). It defers all computation until query time, computing distances to every stored point upon inference.

🎯 **Verified Result:** Question 11 step-by-step resolution per MMDU university marking scheme.

---

### 📘 Question 12 (1 Mark): The Apriori Principle in Frequent Itemset Mining

**Question:**
State the anti-monotonicity property of the Apriori principle that enables search space pruning.

> 📊 **Question Schematic / State Specification:**

```
Question Pruning Lattice: If itemset (Beer, Diaper) is Infrequent ➔ Superset (Beer, Diaper, Milk) is guaranteed Infrequent!
```

---

**Step-by-Step Solution:**

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

```
Solution Anti-Monotone Lattice Formulation:

        `Support(X &amp;cup; Y) ≤ Support(X)`. Therefore: `Support(X) &amp;lt; min\_sup ⟹ Support(X &amp;cup; Y) &amp;lt; min\_sup`.
```

The Apriori Principle states that all non-empty subsets of a frequent itemset must also be frequent. Conversely, if an itemset is infrequent, all of its supersets will inevitably be infrequent, allowing entire sub-lattices to be pruned immediately.

🎯 **Verified Result:** Question 12 step-by-step resolution per MMDU university marking scheme.

---

### 📘 Question 16 (2 Marks): Complete Confusion Matrix Computation

**Question:**
A cancer diagnostic test yields TP = 80, FP = 20, FN = 10, TN = 890. Calculate Precision, Recall, and F1-Score.

> 📊 **Question Schematic / State Specification:**

```
Question 2×2 Contingency Matrix: Actual Positives = 90 (80 TP + 10 FN) | Actual Negatives = 910 (20 FP + 890 TN).
```

---

**Step-by-Step Solution:**

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

```
Solution Performance Metrics Derivation:

        `Precision = TP / (TP + FP) = 80 / (80 + 20) = 80 / 100 = 0.80 (80.0%)`

        `Recall = TP / (TP + FN) = 80 / (80 + 10) = 80 / 90 = 0.8889 (88.89%)`

        `F1-Score = 2 × rac{0.80 × 0.8889}{0.80 + 0.8889} = 1.4222 / 1.6889 = 0.8421 (84.21%)`
```

The model captures ~88.9% of all true cancer cases while maintaining an 80.0% precision rate, resulting in an balanced F1-score of 0.8421.

🎯 **Verified Result:** Question 16 step-by-step resolution per MMDU university marking scheme.

---

### 📘 Question 20 (4 Marks): Information Gain Computation for Decision Tree Splitting

**Question:**
A dataset has 14 samples: 9 Positive (+), 5 Negative (-). An attribute "Wind" splits into: Weak (6+, 2-) and Strong (3+, 3-). Compute Information Gain: Gain(S, Wind).

> 📊 **Question Schematic / State Specification:**

```
Question Partition Tree: Root S [9+, 5-] ➔ Weak [6+, 2-] (8 samples) and Strong [3+, 3-] (6 samples).
```

---

**Step-by-Step Solution:**

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

```
Solution Entropy & Gain Derivation:

        `Entropy(S) = - rac{9}{14} log₂rac{9}{14} - rac{5}{14} log₂rac{5}{14} = 0.9403 bits`

        `Entropy(SWₑₐₖ) = - rac{6}{8} log₂rac{6}{8} - rac{2}{8} log₂rac{2}{8} = 0.8113 bits`

        `Entropy(SSₜᵣₒₙg) = - rac{3}{6} log₂rac{3}{6} - rac{3}{6} log₂rac{3}{6} = 1.0000 bits`

        `Expected Entropy = rac{8}{14} × 0.8113 + rac{6}{14} × 1.0000 = 0.4636 + 0.4286 = 0.8922 bits`

        `Gain(S, Wind) = 0.9403 - 0.8922 = 0.0481 bits`
```

Attribute "Wind" reduces disorder by **0.0481 bits**. The decision tree compares this Gain against all remaining candidate attributes and selects the maximum for the root split.

🎯 **Verified Result:** Question 20 step-by-step resolution per MMDU university marking scheme.

---

## 📘 Question 24 (6 Marks): Mathematical Derivation and Step-by-Step Execution of k-Means Clustering

**Question:**

Given 1D data points X = (2, 4, 10, 12, 3, 20, 30, 11, 25). Cluster into k=2 groups using initial centroids $\mu_1 = 2$ and $\mu_2 = 4$. Perform complete iterations until convergence.

> 📊 **Question Schematic / State Specification:**

```
Question Centroid Displacement Vector: μ₁ = 2, μ₂ = 4 ➔ Assign all points ➔ Recalculate means ➔ Check Δμ = 0.
```

---

**Step-by-Step Solution:**

> ⚡ **KEY TAKEAWAY:**
> **Solution Analytical Derivation & Schematic:**

Solution Iterative Centroid Convergence Trace:

`Iteration 1 Assignment:`

`Distance to $\mu_1(2)$ vs $\mu_2(4)$:`

`Point 2: |2-2|=0 ≤ |2-4|=2 ⟹ Cluster 1`

`Point 3: |3-2|=1 = |3-4|=1 ⟹ Cluster 1 (tie break)`

`Points (4, 10, 11, 12, 20, 25, 30) all closer to $\mu_2$=4 ⟹ Cluster 2`

`Cluster 1 = (2, 3) ⟹ New Centroid $\mu_1$' = (2+3)/2 = 2.5`

`Cluster 2 = (4, 10, 11, 12, 20, 25, 30) ⟹ New Centroid $\mu_2$' = 112 / 7 = 16.0`

`Iteration 2 Assignment ($\mu_1$=2.5, $\mu_2$=16.0):`

`Points (2, 3, 4) closer to 2.5 ⟹ Cluster 1`

`Points (10, 11, 12, 20, 25, 30) closer to 16.0 ⟹ Cluster 2`

`New Centroid $\mu_1$'' = (2+3+4)/3 = 3.0`

`New Centroid $\mu_2$'' = (10+11+12+20+25+30)/6 = 108 / 6 = 18.0`

`Iteration 3 Assignment ($\mu_1$=3.0, $\mu_2$=18.0):`

`Points (2, 3, 4, 10) closer to 3.0? |10-3|=7 vs |10-18|=8 ⟹ Point 10 moves to Cluster 1!`

`Cluster 1 = (2, 3, 4, 10) ⟹ $\mu_1$''' = 19 / 4 = 4.75`

`Cluster 2 = (11, 12, 20, 25, 30) ⟹ $\mu_2$''' = 98 / 5 = 19.6`

`Iteration 4 Assignment ($\mu_1$=4.75, $\mu_2$=19.6):`

`Cluster 1 = (2, 3, 4, 10, 11, 12); Cluster 2 = (20, 25, 30)`

`$\mu_1$ = 42/6 = 7.0; $\mu_2$ = 75/3 = 25.0`

`Iteration 5 Assignment: Assignments remain identical! NO SHIFT ⟹ CONVERGENCE ACHIEVED!`

**Final Converged Clusters:**

- **Cluster 1:** `(2, 3, 4, 10, 11, 12)` with final mean centroid $\mu_1 = 7.0$ .
- **Cluster 2:** `(20, 25, 30)` with final mean centroid $\mu_2 = 25.0$ .

🎯 **Verified Result:** Question 24 step-by-step resolution per MMDU university marking scheme.